

Lenguaje PL/SQL Oracle – Parte 6

Los Paquetes – Package

¿Qué es un paquete?

Un paquete es un objeto de un esquema que agrupa tipos, variables, constantes, subprogramas, cursores y excepciones de PL/SQL relacionados lógicamente. Un paquete se compila y almacena en la base de datos, donde muchas aplicaciones pueden compartir su contenido. Un paquete es como una aplicación.

Un paquete siempre tiene una especificación que declara los elementos públicos referenciables desde fuera del paquete. La especificación del paquete es la interfaz de programación de aplicaciones.

Un paquete es una agrupación o empaquetado de elementos de código PL/SQL en una unidad de programa con nombre que se almacena en la base de datos. Los paquetes proporcionan una estructura (tanto lógica como física) en la que se pueden organizar los programas y otros elementos de PL/SQL, como cursores, tipos y variables. También ofrecen una funcionalidad significativa, incluyendo la capacidad de ocultar la lógica y los datos, así como de definir y manipular datos "globales" o persistentes en la sesión.

¿Por qué usar paquetes?

El paquete es un elemento potente e importante del lenguaje PL/SQL. De hecho, debería ser el componente fundamental de cualquier aplicación basada en PL/SQL. ¿Qué hace que el paquete sea tan potente e importante? Considere sus ventajas:

- **Agrupar funcionalidades relacionadas.** Una de las ventajas más importantes de un paquete es que puede agrupar funcionalidades relacionadas y ponerlas a disposición a través de una API. En lugar de tener que revisar cientos de miles de procedimientos y funciones independientes, encontrará el paquete que contiene la funcionalidad que necesita (que debería ser fácilmente identificable por su nombre) y luego podrá acceder a los subprogramas que necesita. Y cuando necesite añadir nuevos programas a su aplicación, el paquete ofrece un "hogar" para esa funcionalidad. Sin paquetes, a los desarrolladores les resulta mucho más difícil comprender y mantener su código.
- **Capacidad para ocultar detalles de implementación.** Con un paquete, usted elige qué incluir en la especificación para que todos lo vean y usen, y qué incluir en el cuerpo, que solo es visible para el propietario del paquete. Esta capacidad de ocultar detalles de implementación tiene dos ventajas fundamentales. En primer lugar, puede cambiar la implementación sin modificar la especificación.
Esto significa que todos los programas que invocan procedimientos y funciones del paquete no tienen que cambiar. Esta característica aumenta considerablemente la flexibilidad para mejorar su código, a la vez que limita el impacto en la aplicación. En segundo lugar, puede ocultar detalles de implementación que no desea que los usuarios del paquete conozcan.
- **Rendimiento mejorado.** La primera vez que se invoca un elemento de un paquete, Oracle Database carga el paquete completo en memoria. Las referencias posteriores a otros elementos del mismo paquete no requieren E/S de disco adicionales. Además, las variables a nivel de paquete se pueden utilizar como caché a nivel de sesión, lo que ofrece diversas maneras de reducir el tiempo de acceso a los datos.
- **Recompilación de unidades de programa minimizada.** Los programas externos (no definidos en el paquete) solo pueden llamar a los subprogramas enumerados en la especificación. Si se modifica y recompila el cuerpo del paquete, esos programas externos no se invalidarán. Minimizar la necesidad de recompilar código es un factor crucial al administrar grandes volúmenes de lógica de aplicación.

Existen muchas razones para usar paquetes, pero primero es necesario saber cómo construirlos.

La mayoría de las aplicaciones basadas en PL/SQL se construyen a partir de cientos de miles de líneas de código (y algunas llegan a millones) que satisfacen requisitos de usuario detallados y en constante evolución. La lógica de negocio se implementa principalmente en procedimientos y funciones, pero los desarrolladores de PL/SQL deben decidir dónde ubicar dichos procedimientos y funciones.

Por esto es importante saber cómo usar los paquetes para organizar y exponer la funcionalidad de una aplicación de forma que facilite su creación, mantenimiento y optimización del código base.

Un paquete PL/SQL tiene dos partes principales: la **especificación** y el **cuerpo**. La especificación define la interfaz pública del paquete, incluyendo los tipos, variables, constantes, excepciones, cursores y subprogramas (procedimientos y funciones) que pueden ser referenciados fuera del paquete. El cuerpo contiene la implementación real de los subprogramas declarados en la especificación, así como cualquier lógica o código interno que no debe ser visible desde el exterior.

Especificación:

- Es la interfaz pública del paquete, define lo que está disponible para ser utilizado por otras partes del código.
- Se declara utilizando la sentencia CREATE PACKAGE.
- Incluye declaraciones de:
 - Tipos de datos (como TYPE para colecciones o registros).
 - Variables y constantes.
 - Excepciones.
 - Cursores.
 - Subprogramas (procedimientos y funciones).

Cuerpo:

- Contiene la implementación real de los procedimientos y funciones declarados en la especificación.
- El cuerpo puede ser opcional en algunos casos, especialmente si los subprogramas no requieren lógica adicional.
- Se declara utilizando la sentencia CREATE PACKAGE BODY.

En resumen, la especificación define lo que se puede ver y usar del paquete, mientras que el cuerpo contiene el código real que se ejecuta al llamar a esos elementos de la especificación.

Si los elementos públicos incluyen cursores o subprogramas, el paquete también debe tener un cuerpo. Este debe definir consultas para cursores públicos y código para subprogramas públicos. También puede declarar y definir elementos privados referenciables desde fuera del paquete, pero necesarios para su funcionamiento interno.

Finalmente, el cuerpo puede tener una parte de inicialización, cuyas sentencias inicializan variables y realizan otros pasos de configuración puntuales, y una parte de gestión de excepciones. Se puede modificar el cuerpo sin modificar la especificación ni las referencias a los elementos públicos; por lo tanto, se puede considerar el cuerpo del paquete como una caja negra.

Tanto en la especificación como en el cuerpo del paquete, se puede asignar un subprograma de paquete a un subprograma externo de Java o C mediante una especificación de llamada, que asigna el nombre del subprograma externo, los tipos de parámetros y el tipo de retorno a sus equivalentes SQL.

Un Simple Ejemplo

Supongamos que la tabla de empleados está definida de la siguiente manera:

EMPLE_ID	NUMBER(38)
APELLIDO	VARCHAR2(30)
NOMBRE	VARCHAR2(50)

Se necesita escribir un procedimiento, `proceso_emple`, que recupere el "nombre completo" (el apellido, seguido de una coma, seguido del nombre) de un empleado y luego realice otras tareas.

```
CREATE OR REPLACE PROCEDURE proceso_emple (p_emple_id IN empleados.id%TYPE)
IS
    l_nombre_completo VARCHAR2(100);
BEGIN
    SELECT apellido || ', ' || nombre INTO l_nombre_completo FROM empleados
        WHERE id = p_emple_id;
    ...
END;
```

Aunque este código puede funcionar bien hoy, contiene varias "bombas de tiempo" que podrían causar problemas en los próximos meses o años, incluyendo las siguientes:

- La longitud de la variable **l_nombre_completo** está fijada en 100. Habría preferido usar %TYPE para declarar la variable, pero el nombre completo es un valor derivado y no hay ninguna columna a la que pueda adjuntar %TYPE. Por lo que 100 sería suficiente hasta que el administrador de la Bases de Datos (DBA) aumente la longitud máxima de cualquiera de estas columnas.
- La fórmula que produce un "nombre completo" se ha incluido dentro de este procedimiento. Y es muy probable que la misma concatenación aparezca una y otra vez en el código. Además ¿qué haríamos si los usuarios nos comunican que han cambiado de opinión y que quieren ver los nombres como el nombre, seguido de un espacio y, a continuación, el apellido en todos los informes y mensajes? Habría que revisar todo el código en busca de todas las construcciones de `apellido || ', ' || nombre`.
- Finalmente, dado que es tan fácil escribir SQL dentro de programas PL/SQL, es probable que la consulta aparezca en muchos lugares de mi aplicación. Repetir sentencias SQL puede dificultar la optimización del rendimiento de mi código y la adaptación a los cambios en los requisitos del usuario.

Sería fantástico poder escribir este procedimiento de forma que se evite la codificación y la repetición, escribiéndolo de una vez (una definición de un tipo de dato de "nombre completo", una representación de la fórmula y una versión de la consulta) y luego invocarlo donde sea necesario.

Un paquete ofrece el repositorio perfecto para este tipo de implementaciones de código único.

```
1 CREATE OR REPLACE PACKAGE empleado_pkg
2 AS
3     SUBTYPE nombre_completo_t IS VARCHAR2 (100);
4
5     FUNCTION nombrec (
6         apellido empleados.apellido%TYPE,
7         nombre empleados.nombre%TYPE)
8         RETURN nombre_completo_t;
9
10    FUNCTION nombrec (emple_id_in IN empleados.id%TYPE)
11        RETURN nombre_completo_t;
12
13 END empleado_pkg;
```

La especificación del paquete **empleado_pkg** enumera básicamente los diferentes elementos de código que estarán disponibles para su uso fuera del paquete. Veamos qué se hace en cada línea:

- En línea 3, se declara un tipo propio de dato, llamado **nombre_completo_t**, que es un subtipo del tipo VARCHAR2.
- En las líneas 5 a 8, se define una función llamada **nombrec** que acepta un apellido y un nombre, y devuelve el nombre completo. Tener en cuenta que la fórmula en sí (apellido, coma, nombre) no está visible en la especificación del paquete estará en el cuerpo del paquete.
- En la línea 10, se declara una segunda función, también llamada **nombrec**. Esta versión acepta una clave principal para un empleado y devuelve su nombre completo. Tener dos funciones con el mismo nombre es un ejemplo de sobrecarga, que se verá con más detalle a lo largo del apunte.
- Aunque aún no he implementado las dos funciones en esta especificación del paquete, ahora puedo revisar el procedimiento **proceso_emple** y ver cómo puedo usar estos elementos empaquetados, a los que se hace referencia con el formato

```
nombre_paquete.nombre_elemento
```

Como se puede ver a continuación se usa el subtipo para declarar la variable **l_nombre** y se llama a la función en lugar de escribir una consulta directamente dentro de la función.

```
CREATE OR REPLACE PROCEDURE proceso_emple (p_emple_id IN empleados.id%TYPE)
IS
  l_nombre empleado_pkg.nombre_completo_t;
BEGIN
  l_nombre := empleado_pkg.nombrec (p_emple_id);
  ...
END;
```

Como resultado, el código que implementa la fórmula y la consulta SQL se ha eliminado de mi procedimiento; el código resultante es más limpio y sencillo. Lo mismo ocurrirá con todos los demás programas que necesiten trabajar con el nombre completo de un empleado.

```
l_nombre := empleado_pkg.nombrecompleto (p_emple_id);
```

Lo mejor de todo es que, si se necesita cambiar la fórmula para **nombrec** o ampliar el tamaño máximo del tipo de dato **nombre_completo_t**, simplemente se realizará el cambio correspondiente en el paquete y se lo volverá a compilar. Sin embargo, no será necesario modificar los programas que usan los elementos empaquetados. Veamos cómo podría ser el cuerpo de este paquete:

```
1 CREATE OR REPLACE PACKAGE BODY empleado_pkg
2 AS
3   FUNCTION nombrec (p_apellido empleados.apellido%TYPE,
4                     p_nombre empleados.nombre%TYPE)
5   RETURN nombre_completo_t IS
6   BEGIN
7     RETURN p_apellido || ', ' || p_nombre;
8   END;
9
10  FUNCTION nombrec (p_emple_id IN empleados.id%TYPE)
11  RETURN nombre_completo_t IS l_nombrecompleto nombre_completo_t;
12  BEGIN
13    SELECT nombrec(apellido, nombre) INTO l_nombrecompleto
14    FROM empleados WHERE id = p_emple_id;
15    RETURN l_nombrecompleto;
16  END;
17 END empleado_pkg;
```

Las líneas 3 a 8 implementan la primera función **nombrec**, que en realidad no es más que una envoltura de la fórmula apellido-coma-nombre.

Las líneas 10 a 16 implementan la segunda función **nombrec**, que obtiene el nombre y apellido del empleado especificado y los combina llamando a la primera función **nombrec**.

Ahora, si los usuarios solicitan "Nombre-espacio-apellido" sólo se tendrá que cambiar el código del cuerpo del paquete reemplazando la coma por un espacio en "|| ', ' ||" volviendo a compilar el cuerpo del paquete.

También podríamos utilizar las funciones del paquete dentro de una sentencia SQL:

```
select empleado_pkg.nombree(id) from empleados;
```

Datos a nivel de paquete

Los datos a nivel de paquete consisten en variables y constantes que se definen a nivel de paquete; es decir, no dentro de una función o procedimiento específico del paquete. La siguiente especificación de paquete, por ejemplo, declara una variable y una constante a nivel de paquete:

```
CREATE OR REPLACE PACKAGE plsql_limites
IS
    c_varchar2_longitud CONSTANT PLS_INTEGER := 32767;
    g_tiempo_inicio PLS_INTEGER;
END;
```

Cuando se declara una variable dentro de un subprograma o un bloque, se trata de una variable local. Su alcance se limita a ese bloque y, al finalizar este, se libera la memoria de esa variable.

Sin embargo, el alcance de los datos del paquete no es un programa o bloque individual, sino el paquete en su conjunto. En la arquitectura de ejecución de PL/SQL, las estructuras de datos del paquete persisten (conservan sus valores) durante una sesión (en lugar que durante la ejecución de un programa específico).

Si los datos del paquete se declaran dentro del cuerpo del paquete, persistirán durante la sesión, pero solo podrán acceder a ellos los elementos definidos en el propio paquete (datos privados). Si, por el contrario, se declaran dentro de la especificación del paquete, persistirán durante la sesión y serán directamente accesibles (tanto para lectura como para modificación) por cualquier programa con permiso EXECUTE sobre ese paquete (serán datos públicos).

Veamos cómo pueden serle útiles los datos a nivel de paquete. El paquete DBMS_UTILITY ofrece una función llamada GET_CPU_TIME que permite calcular el tiempo transcurrido del código con una precisión de centésimas de segundo. El siguiente bloque anónimo muestra cómo usar esta función para determinar cuánto tiempo se tarda en no hacer nada 10 000 veces.

```
DECLARE
    l_inicio PLS_INTEGER;
BEGIN
    l_inicio := DBMS_UTILITY.get_cpu_time;

    FOR indice IN 1 .. 10000
    LOOP
        NULL;
    END LOOP;

    DBMS_OUTPUT.put_line (DBMS_UTILITY.get_cpu_time - l_start);
END;
/
```

Es bastante sencillo, pero también significa que un desarrollador que desee aprovechar esta utilidad debe declarar una variable local para almacenar la hora de inicio, recordar y escribir la fórmula (fin menos inicio) y luego mostrar el valor. Sin duda, la mayoría de los desarrolladores tienen mejores maneras de invertir su tiempo. Quizás un paquete podría ser útil. Considere, por ejemplo, el paquete de temporizador (timer_pkg).

```
CREATE OR REPLACE PACKAGE timer_pkg
IS
  PROCEDURE tiempo_inicio;

  PROCEDURE mostrar_lapso (p_mensaje IN VARCHAR2 := NULL);
END timer_pkg;
/
```

```
CREATE OR REPLACE PACKAGE BODY timer_pkg
IS
  g_inicio  NUMBER := NULL;

  PROCEDURE tiempo_inicio
  IS
  BEGIN
    g_inicio := DBMS_UTILITY.get_cpu_time;
  END;
  PROCEDURE mostrar_lapso (p_mensaje IN VARCHAR2 := NULL)
  IS
  BEGIN
    DBMS_OUTPUT.put_line (p_mensaje || ' : ' || TO_CHAR (DBMS_UTILITY.get_cpu_time- g_inicio));

    tiempo_inicio;
  END;
END timer_pkg;
/
```

Antes de explicar la implementación de timer_pkg, veamos cómo el paquete facilita a los desarrolladores el uso de la utilidad GET_CPU_TIME.

```
BEGIN
  timer_pkg.tiempo_inicio;
  FOR indx IN 1 .. 10000
  LOOP
    NULL;
  END LOOP;
  timer_pkg.mostrar_lapso ('10000 Nada');
END;
/
```

No es necesario declarar una variable local ni comprender el funcionamiento de la función GET_CPU_TIME (ni siquiera su existencia). Simplemente se llama a los subprogramas correspondientes y se comprueba el rendimiento del código. El paquete timer_pkg facilita la visualización de un mensaje con la temporización y el restablecimiento automático de la hora de inicio tras mostrar el tiempo transcurrido.

El paquete `timer_pkg` se basa en una única variable a nivel de paquete, `g_inicio`, para recordar la hora de inicio entre llamadas a subprogramas del paquete. Esto es posible únicamente gracias a la persistencia de los datos del paquete a nivel de sesión.

Este paquete de temporizador también es un buen ejemplo de API, un conjunto de subprogramas que ofrece una interfaz con la funcionalidad subyacente. La especificación del paquete enumera las tareas que puede realizar, y el cuerpo del paquete oculta la forma en que se implementa el temporizador, aprovechando, en este caso, la función `GET_CPU_TIME`. Si algún día Oracle desarrolla un mecanismo mejor para realizar este tipo de análisis del tiempo transcurrido, solo tendré que cambiar el cuerpo del paquete y todos mis usuarios podrán aprovechar al instante esta técnica nueva y mejorada.

Sobrecarga de Subprogramas

La sobrecarga se produce cuando hay dos o más subprogramas con el mismo nombre en una unidad de programa. Se pueden sobrecargar subprogramas en cualquier sección de declaración; es decir, esta función no sólo está disponible en los paquetes. Sin embargo, observará que casi toda la sobrecarga ocurre dentro de los paquetes. Esto se debe a que la sobrecarga es especialmente útil en las API.

La sobrecarga puede simplificar enormemente su trabajo y el de otros desarrolladores. Al crear una API, los desarrolladores a menudo se encuentran con que necesitan ejecutar el mismo tipo de lógica, pero para diferentes tipos de datos o diversos escenarios que requieren diferentes entradas. Sin sobrecarga, tendría que crear un nombre diferente para cada variación y los usuarios de su código tendrían que recordar cuál es cuál.

Con la sobrecarga, los usuarios de su paquete solo necesitan conocer el nombre que describe la funcionalidad necesaria. El compilador realiza el resto del trabajo: analiza los argumentos pasados al subprograma y los relaciona con una sobrecarga.

La Base de Datos Oracle incluye el procedimiento `DBMS_OUTPUT.PUT_LINE` para enviar texto a la salida del sistema. Este procedimiento no está sobrecargado. Existe una única implementación que acepta un único valor `VARCHAR2`. Por lo tanto, si se pasa una expresión a `DBMS_OUTPUT.PUT_LINE` que evalúa un valor `VARCHAR2`, funcionará correctamente.

```
BEGIN
  DBMS_OUTPUT.PUT_LINE (100);
END;
```

Dado que los números, por ejemplo, se convierten implícitamente en una cadena, la siguiente sentencia mostrará "100". Si, por el contrario, intenta mostrar una expresión booleana, recibirá un mensaje de error.

Esto se debe a que PL/SQL no admite conversiones implícitas de tipos de datos booleanos a `VARCHAR2`. Por ello, muchos desarrolladores escriben código como este cuando quieren mostrar un valor booleano:

```
DBMS_OUTPUT.PUT_LINE ('TRUE');
```

Veamos cómo usar la sobrecarga para facilitar la visualización de información en pantalla desde un bloque PL/SQL. Si se desea facilitar la visualización de valores booleanos y de fecha que incluyan la marca de tiempo (una conversión implícita de una fecha a una cadena usando la máscara de formato predeterminada ignora la hora) sin sobrecarga, se necesita crear diferentes nombres de procedimiento para cada tipo de dato.

No es tan difícil encontrar los nombres de los procedimientos, pero recordar o buscar el nombre correcto es mucho más complicado para el usuario de esta funcionalidad. El procedimiento para mostrar una fecha se llama `"put_date"`, `"put_datetime"` o `"put_timestamp"`, o tal vez en castellano.

Sin embargo, con la sobrecarga, los procedimientos de especificación de un paquete son más fáciles de recordar o buscar:

```
CREATE OR REPLACE PACKAGE salida
IS
  PROCEDURE put_line (value_in IN VARCHAR2);

  PROCEDURE put_line (value_in IN BOOLEAN);

  PROCEDURE put_line (value_in IN DATE,
    mask_in IN VARCHAR2 DEFAULT 'YYYY-MM-DD HH24:MI:SS');
END salida;
/
```

El listado incluye tres procedimientos, todos con el mismo nombre, porque hacen lo mismo: mostrar datos. El usuario sólo tiene que recordar que se llama a un procedimiento para dar salida a una línea **salida.put_line**. Se le pasa los datos que se quiere mostrar y el compilador PL/SQL determina automáticamente cuál de las sobrecargas es la correcta.

Existen reglas y restricciones para la sobrecarga. Algunos puntos clave que se debe tener en cuenta al comenzar a crear los paquetes propios son:

- Las sobrecargas casi siempre involucran subprogramas con diferente cantidad o distintos tipos de parámetros, como se vio en el paquete **salida**.
- Se puede sobrecargar una función y un procedimiento con la misma lista de parámetros.
- Se debe asegurar que los tipos de datos de los parámetros en las sobrecargas no sean demasiado similares; Por ejemplo, no se pueden sobrecargar dos procedimientos cuya única diferencia sea que uno usa un parámetro CHAR y el otro un parámetro VARCHAR2.

Al crear módulos sobrecargados, se dedica más tiempo al diseño y la implementación que con módulos independientes. Este tiempo inicial adicional se verá recompensado con creces a largo plazo, ya que los usuarios encontrarán mucho más fácil usar los programas.

Los Componentes Básicos de las Aplicaciones

Es posible crear aplicaciones en PL/SQL sin usar paquetes. En su lugar, se podrían crear miles de funciones y procedimientos a nivel de esquema. Sin embargo, si se opta por este camino, pronto se descubrirá que es difícil encontrar el código y aún más difícil modificar el código existente en las aplicaciones.

Es recomendable que, en su lugar, siempre se incluyan procedimientos y funciones dentro de paquetes y que se traslade a los paquetes la mayor cantidad posible de código de disparadores y tipos de objeto. Así, la extensa base de código será más fácil de gestionar, ya sea para analizar el impacto de un cambio propuesto o para aplicar una corrección con un impacto mínimo en el código existente.

Se debe aprovechar al máximo la sobrecarga para simplificar la API que presenta a los usuarios del paquete.

Instanciación e inicialización de paquetes

Cuando una sesión hace referencia a un elemento de un paquete, Oracle instancia el paquete para esa sesión. Cada sesión que hace referencia a un paquete tiene su propia instancia de dicho paquete.

Cuando Oracle Database instancia un paquete, lo inicializa. La inicialización incluye cualquiera de las siguientes opciones:

- Asignar valores iniciales a constantes públicas
- Asignar valores iniciales a variables públicas cuyas declaraciones los especifican
- Ejecutar la parte de inicialización del cuerpo del paquete

Estado del Paquete

Los valores de las variables, constantes y cursores que un paquete declara (ya sea en su especificación o en el cuerpo) conforman su estado. Si un paquete PL/SQL declara al menos una variable, constante o cursor, el paquete tiene estado; de lo contrario, no tiene estado. Cuando una sesión utiliza un paquete con estado, Oracle utiliza la memoria específica de la sesión para almacenar los valores de esa sesión para todos los datos a nivel de paquete.

Cada sesión que hace referencia a un elemento del paquete tiene su propia instancia de ese paquete. Si el paquete tiene estado, la instanciación incluye su estado. El estado del paquete persiste durante la sesión, excepto en las siguientes situaciones:

- **El paquete es `SERIALLY_REUSABLE`.** Los paquetes `SERIALLY_REUSABLE` permiten diseñar aplicaciones que gestionan mejor la memoria para una mayor escalabilidad.

Si un paquete no es `SERIALLY_REUSABLE`, su estado se almacena en el área global del usuario (UGA) para cada usuario. Por lo tanto, la cantidad de memoria UGA necesaria aumenta linealmente con el número de usuarios, lo que limita la escalabilidad. El estado del paquete puede persistir durante la duración de una sesión, bloqueando la memoria UGA hasta que esta finaliza. En algunas aplicaciones, como Oracle Office, una sesión típica dura varios días.

Si un paquete es `SERIALLY_REUSABLE`, su estado se almacena en un área de trabajo en un pequeño pool del área global del sistema (SGA). El estado del paquete persiste sólo durante una llamada al servidor. Tras la llamada al servidor, el área de trabajo vuelve al pool. Si una llamada al servidor posterior hace referencia al paquete, Oracle reutiliza una instancia del pool. Al reutilizar una instancia, se reinicializa; por lo tanto, los cambios realizados en el estado del paquete en llamadas al servidor anteriores son invisibles.

- **El cuerpo del paquete se recompila.** Si se recompila el cuerpo de un paquete instanciado con estado (ya sea explícitamente, con la instrucción "ALTER PACKAGE", o implícitamente), la siguiente invocación de un subprograma del paquete hace que Oracle Database descarte el estado del paquete existente y genere la excepción ORA-04068.

Después de que PL/SQL genere la excepción, una referencia al paquete hace que Oracle Database lo vuelva a instanciar, lo que lo reinicializa. Por lo tanto, se pierden los cambios previos en el estado del paquete. (Para obtener información sobre la inicialización, consulte "Instanciación e inicialización de paquetes").

- **Cualquier paquete instanciado de la sesión se invalida y se revalida.** Todas las instanciaciones de paquetes de una sesión (incluidos los estados de los paquetes) pueden perderse si alguno de los paquetes instanciados de la sesión se invalida y se revalida.

El Error ORA-04068

Si se recompila un paquete con estado, todas las sesiones que lo han utilizado fallarán con el error ORA-04068 la próxima vez que intenten acceder a él. Esto se debe a que el estado del paquete en cada sesión está desactualizado, por lo que el paquete debe reinicializarse.

Además, una vez que se genera el error ORA-04068, el estado de todos los paquetes de la sesión (incluido, por ejemplo, `DBMS_OUTPUT`) queda sin inicializar. Esto generalmente significa que el usuario debe desconectarse de la sesión y volver a conectarse para poder usar la aplicación correctamente.

Por ello, a partir de Oracle Database 11g Release 2, Oracle ofrece la función de redefinición basada en ediciones. Con esta función, las organizaciones pueden aplicar parches en caliente a sus aplicaciones, incluso si esto implica cambios en las estructuras de las tablas y en el código. Los usuarios ya no necesitan desconectarse mientras se realizan las actualizaciones.

Directrices para la Escritura de Paquetes

Antes de comenzar a escribir paquetes conozca cuáles son los paquetes que proporciona Oracle a fin de evitar escribir paquetes que dupliquen sus funciones.

Mantenga sus paquetes generales para que las futuras aplicaciones puedan reutilizarlos.

Diseñe y defina las especificaciones de los paquetes antes de los cuerpos de estos.

En las especificaciones de los paquetes, declare únicamente los elementos que deben ser visibles para los programas que los invocan. Esta práctica evita que otros desarrolladores creen dependencias inseguras en los detalles de su implementación y reduce la necesidad de recompilación.

Si modifica la especificación del paquete, debe recompilar cualquier subprograma que invoque los subprogramas públicos del paquete. Si solo modifica el cuerpo del paquete, no es necesario recompilar esos subprogramas.

Declare los cursores públicos en las especificaciones de los paquetes y definalos en los cuerpos de los paquetes. Esta práctica le permite ocultar las consultas de los cursores a los usuarios del paquete y modificarlos sin modificar las declaraciones de los cursores.

Asignar valores iniciales en la parte de inicialización del cuerpo del paquete en lugar de en las declaraciones. Esta práctica tiene las siguientes ventajas:

- El código para calcular los valores iniciales puede ser más complejo y estar mejor documentado.
- Si el cálculo de un valor inicial genera una excepción, la parte de inicialización puede gestionarla con su propio gestor de excepciones.

Ejercicios de Paquetes

1. Crear un paquete que permita:
 - a. Mediante un procedimiento llamado **datos**, de un empleado se muestre el apellido, el nombre y el mail al indicarle su legajo.
 - b. Mediante una función llamada **antigüedad** al recibir el legajo de un empleado entregue su antigüedad en la empresa.
 - c. Mediante una función llamada **nomcompleto** al recibir el legajo de un empleado entregue su Apellido y su Nombre separados por coma.
2. Escribir un paquete completo para gestionar los departamentos. El paquete se llamará **gestion_dpto** y deberá incluir, al menos, los siguientes subprogramas:
 - a. **insertar**: permite insertar un departamento nuevo. El procedimiento recibe el nombre y la localidad del nuevo departamento. Creará el nuevo departamento comprobando que el nombre no se duplique y le asignará como número de departamento la decena siguiente al último número de departamento utilizado.
 - b. **borrar**: permite borrar un departamento. El procedimiento recibirá dos números de departamento de los cuales el primero corresponde al departamento que queremos borrar y el segundo al departamento al que pasarán los empleados del departamento que se va a

eliminar. El procedimiento se encargará de realizar los cambios oportunos en los números de departamento de los empleados correspondientes.

- c. **modificar**: modifica la localidad del departamento. El procedimiento recibirá el número del departamento a modificar y la nueva localidad, y realizará el cambio solicitado.
 - d. **visualizar**: visualizará los datos de un departamento cuyo número se pasará en la llamada. Además de los datos relativos al departamento, se visualizará el número de empleados que pertenecen actualmente al departamento.
 - e. **visualizar**: versión sobrecargada del procedimiento anterior que, en lugar del número del departamento, recibirá el nombre del departamento. Realizará una llamada a la función **buscar**, función local.
 - f. **buscar**: función local del paquete. Recibe el nombre de un departamento y devuelve el número de este.
3. Desarrollar un paquete que permita darle mantenimiento a la tabla departamentos (crear un procedimiento para cada acción de actualización).
4. Escribir un paquete completo para gestionar los empleados. El paquete se llamará **gest_emple** e incluirá, al menos los siguientes subprogramas:
- a. **insertar_emple**
 - b. **borrar_emple**. Cuando se borra un empleado todos los empleados que dependían de él pasarán a depender del jefe del empleado borrado.
 - c. **modificar_categoria_emple**
 - d. **modificar_dept_emple**
 - e. **modificar_dire_emple**
 - f. **modificar_salario_emple**
 - g. **modificar_comision_emple**
 - h. **visualizar_emple**: También se incluirá una versión sobrecargada del procedimiento que recibirá el nombre del empleado.
 - i. **buscar_emple_por_dnombre**: Función local que recibe el nombre y devuelve el número.
- Todos los procedimientos recibirán el número del empleado seguido de los demás datos necesarios.
5. Incluir en el paquete del ejercicio 4. cursores y declaraciones de tipo registro, así como siguientes procedimientos que afectarán a todos los empleados:
- a. **subida_salario_porcen**: incrementará el salario de todos los empleados el porcentaje indicado en la llamada que no podrá ser superior al 25%.
 - b. **subida_salario_imp**: sumará al salario de todos los empleados el importe indicado en la llamada. Antes de proceder a la incrementar los salarios se comprobará que el importe indicado no supera el 25% del salario medio.
 - c. **Calculo_sueldo_aportes**: al total de los salarios del mes le calculará los aportes patronales el 16% para jubilación, el 6% para Obra Social, el 2% para PAMI y 1,5% para fondo de desempleo. Devolver el monto total a desembolsar en concepto de sueldos más de aportes por todos los empleados de la nómina.